

DSA & Advanced DSA Roadmap

Phase 10 — Backtracking

Learning Objectives

By the end of this phase, learners will be able to:

- Explain backtracking as depth-first search over a decision tree, where invalid or exhausted branches are abandoned ('backtracked') to try the next choice.
- Identify the three components of any backtracking solution: the choice at each step, the constraint that prunes invalid choices, and the goal/base case that signals a complete solution.
- Apply pruning to cut off branches early rather than generating every possibility and filtering afterward.
- Recognize the five classic backtracking shapes: subsets, permutations, combinations, constraint-satisfaction grids (N-Queens), and constraint-satisfaction puzzles (Sudoku).
- Solve Easy to Medium level backtracking problems independently, and approach Hard constraint-satisfaction problems (N-Queens, Sudoku Solver) by identifying the placement, validity check, and backtrack steps.

Implementation & Internals

Core Concepts

- Decision Trees — modeling a backtracking problem as a tree where each node is a partial solution and each edge is one choice; the algorithm performs DFS over this tree.
- State Space Search — revisiting Phase 9's state-space framing, now with explicit pruning: abandon a branch as soon as it's known to be invalid, rather than exploring it to completion.

Implementation Exercises

- Implement a generic backtracking template with three named steps — choose, explore, un-choose — and instantiate it for at least two different problems (e.g. subsets and permutations) to see the shared structure.
- Add a pruning condition to a naive backtracking solution (e.g. for Combination Sum) and measure how many fewer recursive calls are made compared to the unpruned version on the same input.
- Trace the decision tree by hand for generating all subsets of a 3-element set, and count the leaves to confirm they match 2^3 .

Pattern Mastery

Pattern 1: Subsets

Concepts

- Building every possible subset of a set by making an include/exclude choice for each element.
- Handling duplicate elements in the input by sorting first and skipping over repeated choices at the same recursion depth.

Practice Problems

- Given an array of unique integers, return all possible subsets.
- Given an array that may contain duplicates, return all possible subsets without duplicate subsets in the output.

LeetCode

- [Subsets — leetcode.com/problems/subsets](https://leetcode.com/problems/subsets)
- [Subsets II — leetcode.com/problems/subsets-ii](https://leetcode.com/problems/subsets-ii)

Pattern 2: Permutations

Concepts

- Building every possible ordering of a set by choosing, at each step, any not-yet-used element to place next.
- Tracking which elements have already been used in the current partial permutation, typically with a boolean array or set.

Practice Problems

- Given an array of distinct integers, return all possible permutations.
- Given a digit string, return all possible letter combinations the digits could represent on a phone keypad (a permutation-flavored backtracking problem over a fixed mapping rather than the input array itself).

LeetCode

- [Permutations — leetcode.com/problems/permutations](https://leetcode.com/problems/permutations)
- [Letter Combinations of a Phone Number — leetcode.com/problems/letter-combinations-of-a-phone-number](https://leetcode.com/problems/letter-combinations-of-a-phone-number)

Pattern 3: Combinations

Concepts

- Building every fixed-size selection from a set where order doesn't matter, using a start index to avoid generating the same combination twice in different orders.
- Extending plain combinations to combinations that must also sum to a target, pruning branches once the running sum exceeds the target.

Practice Problems

- Given two integers n and k , return all possible combinations of k numbers chosen from the range $[1, n]$.
- Given an array of candidate numbers and a target, find all unique combinations that sum to the target, where each number may be reused.
- Given an array of candidate numbers (which may contain duplicates) and a target, find all unique combinations that sum to the target, where each number may be used at most once.

LeetCode

- [Combinations — leetcode.com/problems/combinations](https://leetcode.com/problems/combinations)
- [Combination Sum — leetcode.com/problems/combination-sum](https://leetcode.com/problems/combination-sum)
- [Combination Sum II — leetcode.com/problems/combination-sum-ii](https://leetcode.com/problems/combination-sum-ii)

Pattern 4: N Queens

Concepts

- Placing queens one row at a time, checking column and diagonal conflicts against all previously placed queens before committing to a position.
- Pruning aggressively: a queen placement is checked for validity immediately, rather than placing all n queens and checking the whole board afterward.

Practice Problems

- Given an integer n , place n queens on an $n \times n$ chessboard so that no two queens attack each other, and return all distinct solutions.

LeetCode

- [N-Queens — leetcode.com/problems/n-queens](https://leetcode.com/problems/n-queens)

Pattern 5: Sudoku Solver

Concepts

- Filling empty cells one at a time, checking row, column, and 3×3 sub-box constraints before committing to a digit.
- Backtracking when no digit 1–9 satisfies all three constraints for the current cell, undoing the most recent placement and trying the next candidate.

Practice Problems

- Write a program that solves a given 9×9 Sudoku puzzle by filling in all empty cells, modifying the board in place.

LeetCode

- [Sudoku Solver — leetcode.com/problems/sudoku-solver](https://leetcode.com/problems/sudoku-solver)

Phase Assessment

Implementation Tasks

- Implement the generic choose/explore/un-choose backtracking template from scratch and apply it to at least two distinct problems.
- Add and justify a pruning rule for a chosen backtracking problem (e.g. Combination Sum), demonstrating the reduction in recursive calls.
- Analyze and explain why N-Queens and Sudoku Solver are Hard despite using 'the same' backtracking template as Subsets and Permutations — focus on constraint-checking cost and branching factor.

Recommended LeetCode Target

Easy Problems

None assigned this phase — backtracking problems are inherently combinatorial and rarely rated Easy on LeetCode. The Medium tier below serves as this phase's entry point.

Medium Problems

Subsets

- leetcode.com/problems/subsets
- leetcode.com/problems/subsets-ii

Permutations

- leetcode.com/problems/permutations
- leetcode.com/problems/letter-combinations-of-a-phone-number

Combinations

- leetcode.com/problems/combinations
- leetcode.com/problems/combination-sum
- leetcode.com/problems/combination-sum-ii

Hard Problems

- leetcode.com/problems/n-queens
- leetcode.com/problems/sudoku-solver

On scope and dedup

This phase has 7 distinct problems, with no Easy tier (genuine backtracking problems are essentially never rated Easy on LeetCode) and 2 Hard problems instead of the usual 1, since N-Queens and Sudoku Solver are both named directly in the original phase outline and neither can be swapped for an easier substitute without losing the constraint-satisfaction skill this phase exists to teach. None of the 7 problems overlaps with any problem used in Phases 0–9.

Coding Challenges (Assessment Day)

- Subsets
- Permutations
- Combination Sum
- N-Queens
- Sudoku Solver

Expected Outcome

Students should be able to:

- Recognize a problem as a backtracking problem from its description (generate all..., find all valid arrangements..., place items subject to constraints...).
- Apply the choose/explore/un-choose template to new problems without rederiving it from scratch each time.
- Add effective pruning to avoid exploring branches that cannot lead to a valid solution.
- Solve unseen backtracking problems using pattern-based thinking, and approach Hard constraint-satisfaction problems methodically rather than treating them as qualitatively different from Medium backtracking problems.